Deep Learning Approaches to Jet Tagging in High-Energy Physics

Deniz Yoldas*

Department of Physics, Boston University, Boston, MA 02215

Ecole de Physique, Université de Genève, Genève, 1211

June 16, 2025

Abstract

We employ modern deep learning architectures to classify jets by their origin. Over a dataset of *1,546,865* jets, our binary classifier achieves a balanced accuracy of *80%* in distinguish QCD jets from background, while our multiclass classifier achieves a balanced accuracy of *79%* in separating QCD, Top, and W/Z jets.

1 Introduction

At the Large Hadron Collider (LHC), high-energy proton-proton collisions produce a variety of fundamental particles. Many of these particles are unstable and rapidly decay into multiple hadrons that travel in approximately the same direction, forming collimated sprays known as *jets*. These jets are recorded by detectors and contain rich information about the underlying physics processes.

Accurately identifying the origin of a jet –whether it was initiated by a heavy particle decay or by a light quark or gluon– is a central challenge in collider physics. The goal of this project is to develop machine learning models capable of classifying jets based on their kinematic, substructure, and constituent-level features. Specifically, our aim is to distinguish between three jet classes:

- Jets originating from hadronic decays of W or Z bosons;
- Jets originating from the decay of Top quarks;
- Jets initiated by light quarks or gluons, collectively referred to as QCD jets.

We train deep neural networks on a labeled dataset of jets collected by the ATLAS detector. We investigate the performance of both binary and multiclass classifiers, assess the impact of feature- and class-level weighting and to mitigate biases from kinematic variables, and explore the benefits of using constituent-level input to enhance classification performance.

*Deniz.Yoldas@etu.unige.ch

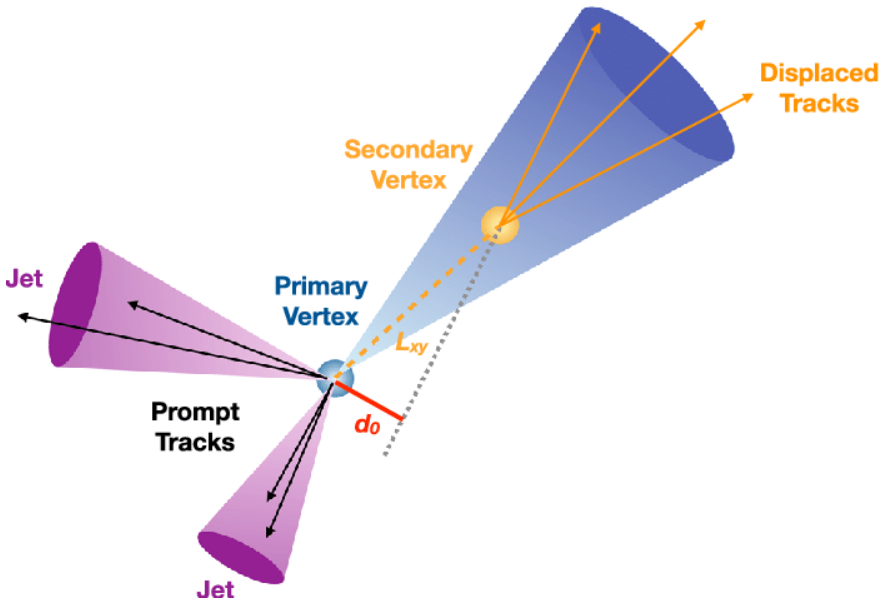


Figure 1: A schematic diagram of an interaction producing two lightflavour jets and one b-jet [2].

2 Dataset

The dataset used in this study is publicly available via SWITCHdrive [1] and contains labeled jet events suitable for both binary and multiclass classification. The samples are distributed as follows:

- 500,000 jets of QCD origin,
- 300,000 jets originating from Top quark decays,
- 746,865 jets originating from W or Z boson decays.

The dataset provides two types of jet representations: *scalar-level variables* and *constituent-level features*, stored in the `#_small.h5` and `#_const.h5` files, respectively.

2.1 Scalar Features

Each jet in the `#_small.h5` files is characterized by a set of 11 scalar features. These include the jet's four-momentum in cylindrical coordinates: Jet mass (m), Transverse momentum (p_T), Pseudorapidity (η), Azimuthal angle (ϕ).

And, additional variables known as *substructures*. *Substructures* describe properties of the jet not directly related to its momentum, for example how the energy is distributed within the jet or whether it has a multiple-prong structure. They are calculated using "constituents" of the jet, which are the energy deposits recorded by the detector which are grouped (clustered) together to build the jet. Kinematic features along with *substructures* form the following scalar features vector for a given jet (See histogram distributions in appendix 15 16):

$$[p_T,\ \eta,\ \phi,\ m,\ \tau_1,\ \tau_2,\ \tau_3,\ d_{12},\ d_{23},\ ECF_2,\ ECF_3]$$

2.2 Preprocessing Scalar Features

In order for the model to learn the shape of scalar feature distributions —rather than relying on their absolute numerical scales— preprocessing is essential [3]. We begin by applying *standard scaling* to each feature, which transforms the values to have zero mean and unit variance:

$$x' = \frac{x - \mu}{\sigma} \tag{1}$$

where x is the original feature value, μ is the mean of the feature over the training set, and σ is the corresponding standard deviation. This normalization step ensures that all input features contribute comparably to the model's learning process, avoiding bias toward features with larger numerical ranges. See Fig 2.

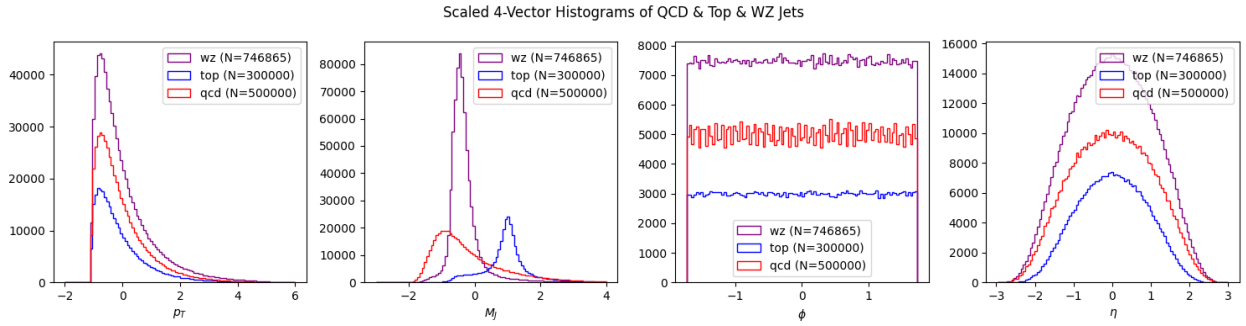


Figure 2: Histogram distributions of scaled 4-vector features of the QCD, Top, & W/Z jets. In the order of p_T, m, ϕ, η. See appendix for substructure histograms 17.

2.3 Constituent-Level Features

The `#_const.h5` files provide low-level constituent information for each jet. Each constituent corresponds to an individual energy deposit in the detector (e.g., a calorimeter cluster or reconstructed particle) and is described by a four-vector: [p_T, η, ϕ, m]. 20 constituents are included per jet.

Constituent-level features retain fine-grained spatial and kinematic structure, enabling the use of expressive architectures such as Deep Sets. These inputs also serve as the basis for constructing jet images and other geometric representations that can enhance classification performance.

2.4 Preprocessing Constituent-Level Features

To calibrate the constituents of each jet, we center their η and ϕ coordinates relative to the jet axis. Additionally, we augment the dataset by computing the constituent energy E and including it as a fifth feature per constituent. The energy is calculated using the transverse momentum and pseudorapidity via the relation:

$$E = p_T \cosh(\eta) \tag{2}$$

This augmentation provides a Lorentz-invariant energy scale per constituent and enables more expressive modeling of constituent-level structure.

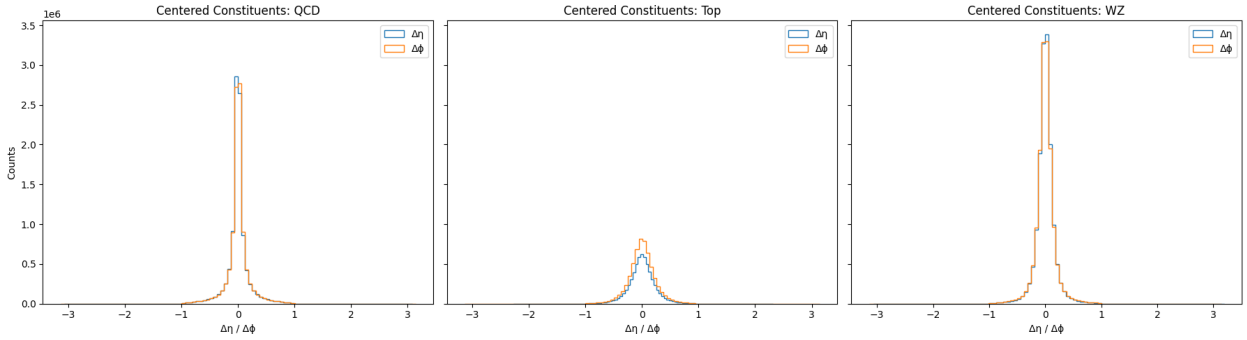


Figure 3: Centered constituent-level η and ϕ for QCD, Top, & W/Z jets. See appendix for a example jet image made using the new calculated E of constituents 18.

2.5 Class-Level Weighting

After preprocessing, one key challenge remains: the dataset exhibits significant *class imbalance*. This imbalance can cause the model to learn shortcuts by biasing predictions toward the majority class, thereby degrading performance on underrepresented classes.

To counteract this, we apply *class-level weighting* during training for both binary and multiclass classification tasks. Class weights are computed using the inverse frequency formula:

$$w_c = \frac{N}{K \cdot n_c} \tag{3}$$

where w_c is the weight for class c, n_c is the number of samples in class c, N is the total number of samples, and K is the number of classes. These weights are incorporated into the loss function, penalizing errors on underrepresented classes more heavily.

To illustrate the effect of bias due to imbalance, we evaluate the loss of a constant classifier that always predicts a fixed class. In the case of binary classification, a completely random classifier yields a cross-entropy loss of approximately 0.693. However, in our imbalanced dataset, even a naive fixed-class predictor achieves a lower loss due to majority class dominance. This local minimum in the loss landscape reflects undesirable behavior that a model could exploit. See Fig 4.

Applying class-level weighting mitigates this risk by guiding the model away from trivial fixed-class predictions and encouraging it to learn meaningful class boundaries across all categories.

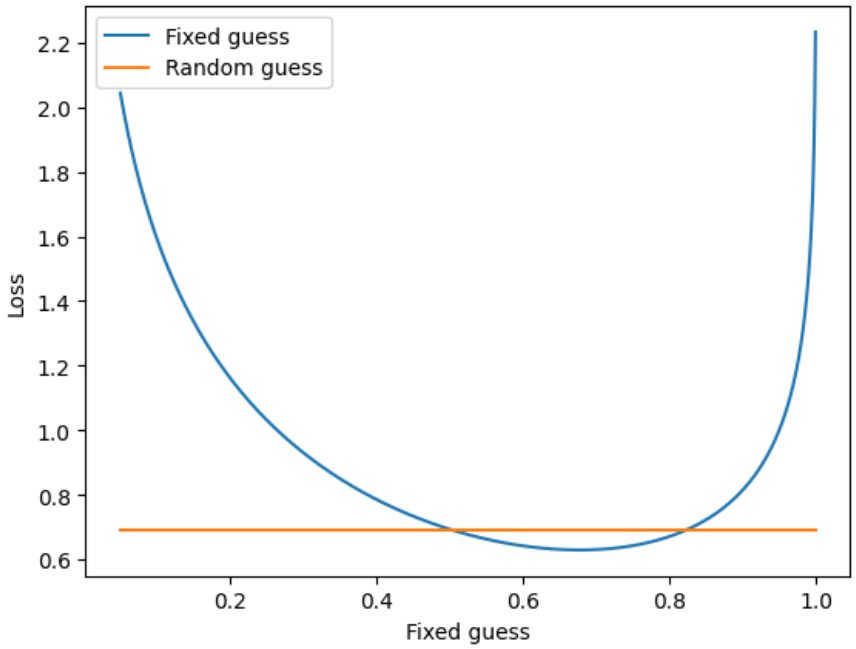


Figure 4: Fixed-class predictor achieves a lower loss due to majority class dominance (QCD vs Top + WZ).

2.6 Feature-Level Weighting & Feature Cutting

As shown in Figure 2, the kinematic features —particularly the jet mass— exhibit strong separation between classes. While this separation can help classification, it introduces an undesirable bias: the model may rely too heavily on explicit kinematic information instead of learning from more generalizable substructure features.

To mitigate this effect, we apply *feature-level reweighting* to the kinematic inputs. Specifically, for each kinematic feature f, we compute a histogram-based weighting function to match the feature distribution between classes. The per-feature weights are computed using the inverse bin ratio between the signal and background distributions:

$$w_f(x) = \frac{1}{\mathrm{PDF}_{\mathrm{class}}(x_f)} \tag{4}$$

where $\mathrm{PDF}_{\mathrm{class}}(x_f)$ denotes the estimated probability density function (e.g., from a histogram) of feature f for the corresponding class. Each jet is assigned a set of scalar weights —one per feature— which are then aggregated into a single per-sample weight using a permutation-invariant mean. These aggregated weights are used to modulate the loss during training, effectively down-weighting samples that reside in high-density kinematic regions and encouraging the model to learn class separation based on substructure. See Fig 5.

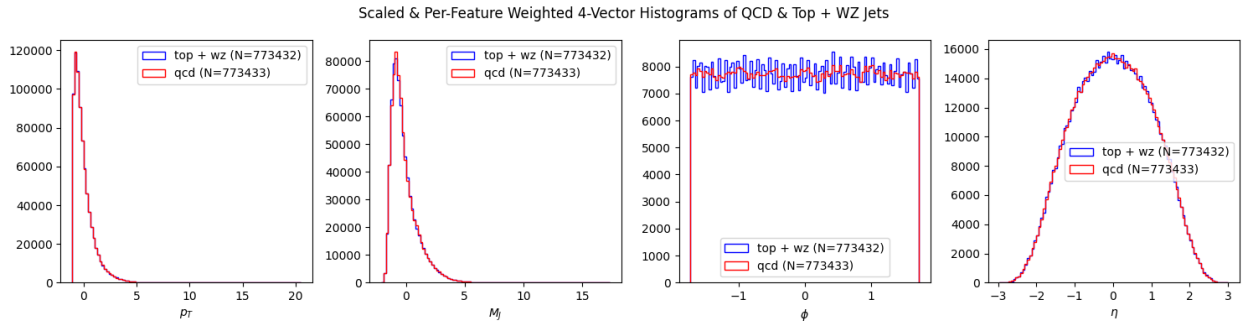


Figure 5: Feature-level weighing applied to signal and background jets (QCD vs Top + W/Z).

In addition to reweighting, we explore a more aggressive strategy: *feature cutting*, where the four-vector components (p_T, η, ϕ, m) are removed entirely from the scalar input. This forces the model to rely purely on substructure observables, allowing us to assess the classification power of shape-sensitive features alone.

3 Model

To process scalar features, we implement a standard Multi-Layer Perceptron (MLP) architecture. For constituent-level features, we construct a Deep Sets architecture to enforce permutation invariance among constituents within a given jet.

3.1 Multi-Layer Perceptron

Our MLP consists of four fully connected layers. Each hidden layer applies the standard sequence: a linear transformation, followed by (optional) batch normalization, a ReLU activation, and (optional) dropout. This ordering improves training stability and generalization by mitigating overfitting and internal covariate shift [4, 5].

The architecture is summarized in Table 1.

Layer	Transformation
Layer 1	Input dimension $\rightarrow$ Hidden dimension
Layer 2	Hidden dimension $\rightarrow$ Hidden dimension
Layer 3	Hidden dimension $\rightarrow$ Hidden dimension
Layer 4	Hidden dimension $\rightarrow$ Output dimension

Table 1: Architecture of the Multi-Layer Perceptron used for scalar feature classification. Each hidden layer applies: Linear $\rightarrow$ (optional BatchNorm) $\rightarrow$ ReLU $\rightarrow$ (optional Dropout).

The input x is a rank-2 tensor with shape (N, F), where N is the number of jets and F is the number of scalar features per jet (typically 11 or 7). In index notation, x_{ij} denotes the j-th scalar feature of the i-th jet.

The forward pass of the MLP can be written as:

$$f(x) = \sigma\big(W_4 \cdot \phi\big(D_3 \cdot \phi\big(B_3 \cdot W_3 \cdot \phi\big(D_2 \cdot \phi\big(B_2 \cdot W_2 \cdot \phi\big(D_1 \cdot \phi\big(B_1 \cdot W_1 \cdot x + b_1\big)\big) + b_2\big)\big) + b_3\big)\big) + b_4\big) \tag{5}$$

Here, W_i and b_i denote the weights and biases of layer i, B_i represents batch normalization, D_i denotes dropout, and $\phi(z) = \max(0, z)$ is the ReLU activation. The output layer applies $\sigma(\cdot)$, which is:

- the **sigmoid** function for binary classification with binary cross-entropy loss,
- or the **softmax** function for multiclass classification with categorical cross-entropy loss.

3.2 Deep Sets

For constituent-level features, the ordering of constituents within a jet has no physical significance. The jet's identity should not depend on how its constituent list is permuted. To respect this symmetry, we adopt the *Deep Sets* framework [6], which ensures that the model is invariant under permutations of input constituents.

The Deep Sets architecture operates on each jet as a set of 20 constituents, where each constituent is a 5-dimensional vector after preprocessing (p_T, η, ϕ, m, E). The input tensor has shape (N, C, F), where N is the number of jets, C is the number of constituents per jet, and $F = 5$ is the number of features per constituent.

Each constituent vector x_{ijk} (jet i, constituent j, feature k) is passed through a shared MLP ϕ to map it into a latent representation. A permutation-invariant aggregation is then applied (e.g., element-wise mean across constituents), and the result is passed to a second MLP ρ for classification:

$$f(x_i) = \rho\left(\frac{1}{C}\sum_{j=1}^{C}\phi(x_{ij})\right) \tag{6}$$

This architecture is summarized in Table 2.

Component	**Operation**
Constituent MLP ϕ	Maps each constituent vector to hidden dimension
Aggregation	Mean over constituent embeddings (permutation-invariant)
Jet-level MLP ρ	Maps aggregated embedding to output (1 or 3)

Table 2: Architecture summary of the Deep Sets model used for constituent-level classification.

This approach allows the model to learn from fine-grained constituent information while preserving the necessary symmetry under permutations. The Deep Sets model is trained using the same loss functions as the MLP model: binary cross-entropy with sigmoid for two-class tasks, and categorical cross-entropy with softmax for multiclass tasks.

3.3 Training the Model

We train all models in a supervised setting using the Adam optimizer with a fixed learning rate. The dataset is split into training (75%), validation (15%), and test (10%) subsets with a fixed random seed to ensure reproducibility.

The loss function is Binary Cross-Entropy (BCE) for binary classification and Cross-Entropy loss for multiclass classification. In both cases, class-level weights are incorporated to address imbalance.

To improve generalization, we apply dropout, batch normalization, and L_2 weight regularization (weight decay). Early stopping is employed based on validation loss to prevent overfitting, and the best model checkpoint is saved for final evaluation.

During training, mini-batch gradient descent is used (batch size B), and performance is monitored on the validation set. Feature-level weighting is optionally applied to decorrelate classification from kinematic features. Final results are reported on the held-out test set using the best-performing model.

4 Binary Classification

For the binary classification task distinguishing QCD jets (background) from Top and W/Z jets (signal), all models were trained with the same hyperparameters: hidden dimension = 128, learning rate = 0.001, batch size = 256, weight decay = 0.01, and 20 training epochs. The four model variants evaluated are:

- A standard scalar-based MLP trained with binary cross-entropy (BCE),
- A class-weighted regular MLP to address class imbalance,
- A constituent-based Deep Sets model with BCE,
- A class-weighted Deep Sets model.

Model	**Precision**	**Recall**	**F1 Score**	**Balanced Accuracy**
Regular BCE	0.861	0.911	0.885	0.800
Regular + Class Weights	0.864	0.905	0.884	0.802
Deep Sets BCE	0.840	0.911	0.874	0.774
Deep Sets + Class Weights	0.832	0.916	0.872	0.766

Table 3: Binary classification performance on test data distinguishing QCD vs Top/W/Z jets. Metrics are reported at the F1-optimized threshold.

A perfect classifier would yield a best threshold of 0.5, with all performance metrics equal to 1. In practice, thresholds shift away from 0.5 when the predicted probabilities of signal and background overlap. The optimal threshold is selected to maximize the F1 score on the validation set.

- **Precision**: Fraction of jets predicted as Top/W/Z (signal) that are correct.

- **Recall**: Fraction of true Top/W/Z jets correctly identified.
- **F1 Score**: Harmonic mean of precision and recall; balances false positives and false negatives.
- **Balanced Accuracy**: Mean of true positive and true negative rates; accounts for class imbalance.

Scalar-based models achieve the highest performance, with F1 scores near 0.89 and balanced accuracy around 0.80. Class weighting yields a marginal gain in balanced accuracy (0.802 vs. 0.800), indicating minor benefit under class imbalance.

Deep Sets models match scalar recall but underperform in precision and balanced accuracy, suggesting that while constituent-level features capture substructure, scalar variables — especially jet mass and p_T — are more directly discriminative.

Overall, scalar inputs offer the clearest separation between QCD and Top/W/Z jets. The Scalar BCE model —trained with QCD as class 0 (background) and Top/W/Z as class 1 (signal)— demonstrates the best performance, with stable training curves, high AUC, and balanced classification confirmed by the confusion matrix. See Fig 6 7 8.

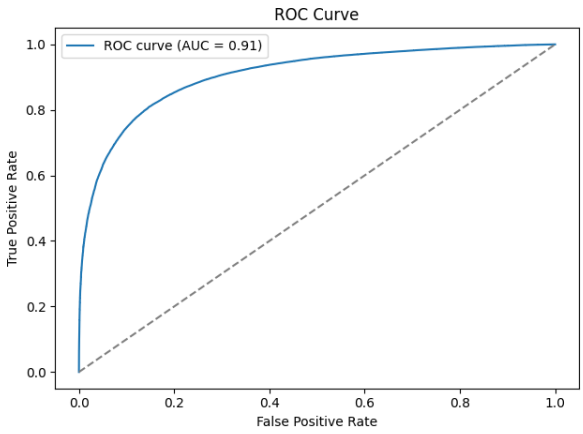


Figure 6: ROC curve of the Scalar BCE model. AUC = 0.91 indicates strong separability between class 0 (QCD) and class 1 (Top/W/Z).

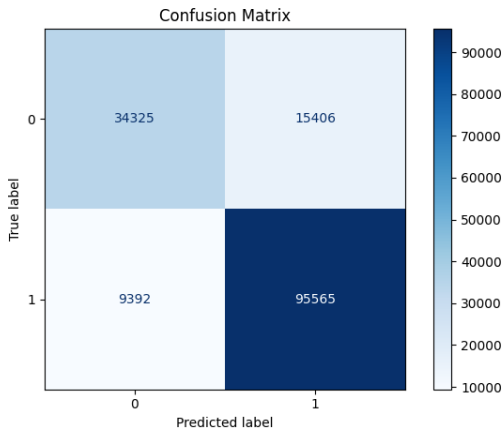


Figure 7: Confusion matrix at the F1-optimized threshold. True positives (Top/W/Z) and true negatives (QCD) dominate, with few misclassifications.

4.1 Effect of Jet Four-Momentum

A key objective is to reduce the classifier's reliance on jet kinematics —particularly jet mass— which strongly separates QCD and Top/W/Z jets. Ideally, the model should base its decisions on substructure information rather than easily separable kinematic variables.

Figure 9 shows the AUC score as a function of jet mass for the best-performing model (Standard BCE; see Table 3). The model performs well overall, but AUC is notably higher

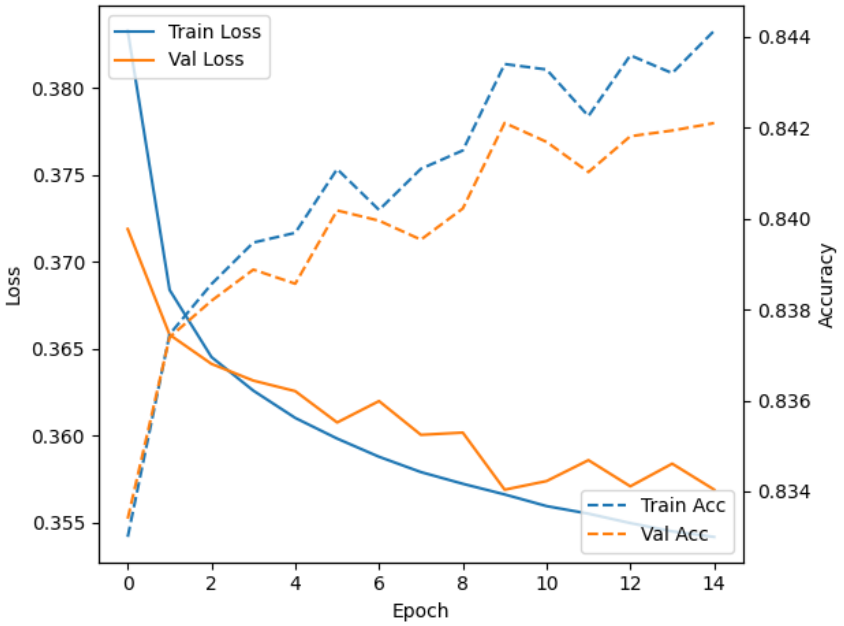


Figure 8: Training and validation loss curves for the Scalar BCE model. Early stop at 14, no overfitting observed.

in mass regions with strong class separation, particularly around 100 and 180 GeV, where distinct peaks are visible in the jet mass distribution (See Fig 15). In contrast, performance declines in both the low-mass and high-mass tails, where signal and background distributions overlap more or data is sparse.

The processing methods described in Section 2.6. —feature-level weighting and four-vector cutting— were applied to the scalar dataset and trained using the regular BCE model. The results are summarized below, with all metrics evaluated at the F1-optimized threshold 4.

Model	**Precision**	**Recall**	**F1 Score**	**Balanced Accuracy**
Class Weighted + Four-Vector Cut	0.831	0.918	0.873	0.762
Feature Weighted	0.861	0.906	0.883	0.798

Table 4: Performance of regular BCE models trained with feature-cutting or feature-weighting applied. Metrics are reported at the F1-optimized threshold.

Both models perform slightly worse than those trained with full four-momentum information (cf. Table 3). However, feature weighting outperforms four-vector cutting. This is expected: although feature-level reweighting suppresses the model's reliance on jet mass and p_T, it still retains embedded information, allowing the network to learn subtle correlations between kinematics and substructure. By contrast, cutting the four-vector entirely removes this informative signal.

The mass-dependent AUC plots in Figures 10, 11 further illustrate these effects.

Surprisingly, cutting the four-vector did not improve the mass-dependent AUC curve,

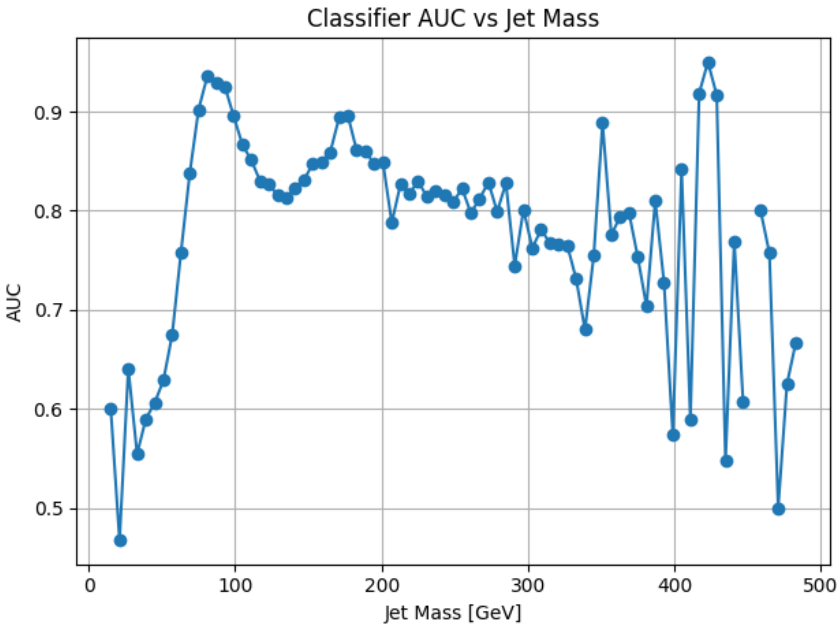


Figure 9: Classifier AUC vs. jet mass for the best performing BCE model. AUC is highest in the intermediate mass range, indicating strong class separability driven by jet mass. Lower AUC at the tails suggests weaker discriminative power or limited data.

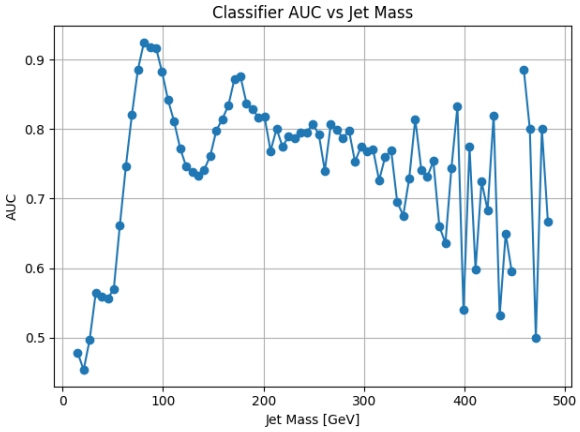


Figure 10: AUC vs. jet mass for class-weighted BCE with four-vector cut. Performance degrades across most mass ranges.

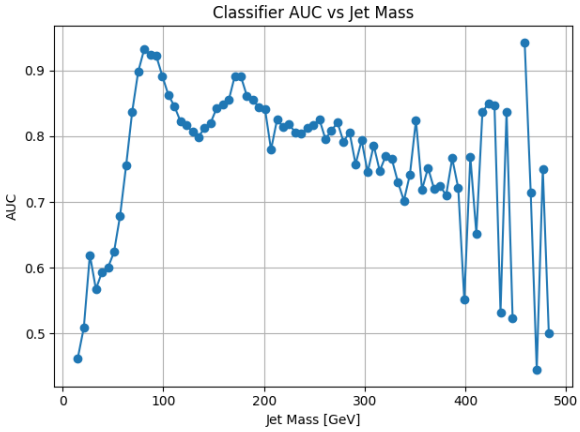


Figure 11: AUC vs. jet mass for BCE with feature-level weighting. Retains consistent gradient.

despite the model being fully blind to obvious jet kinematics. In contrast, although not a significant improvement, feature weighting produced a more consistent AUC gradient across increasing jet mass, suggesting that it reduced sharp kinematic biases without fully discarding useful information. These results highlight that model performance remains noticeably dependent on four-momentum features —even at deeper levels. This is expected, as sub-

structure variables themselves are derived from the constituent-level kinematic data, making it difficult to entirely decorrelate classification performance from jet mass and p_T which is unphysical.

5 Multiclass Classification

The binary classification architecture is extended to multiclass classification by setting the output dimension to 3 and applying a softmax activation, as described in Section 3. Both scalar (small) and constituent-level datasets are analyzed. The results are summarized below:

Model Variant	**Balanced Accuracy**	**Weighted F1 Score**
Regular	0.792	0.801
Scalar + Class Weights	0.787	0.797
Deep Sets	0.777	0.793
Deep Sets + Class Weights	0.765	0.787

Table 5: Multiclass classification performance on test data (QCD, Top, W/Z jets).

Among all models, the regular scalar-based classifier performs best, achieving a balanced accuracy of 0.792 and weighted F1 score of 0.801. Incorporating class weights yields a slight decrease in both metrics. Deep Sets classifiers perform slightly worse overall, again reflecting that scalar features —particularly jet mass— remain highly informative. See Figs 12 13.

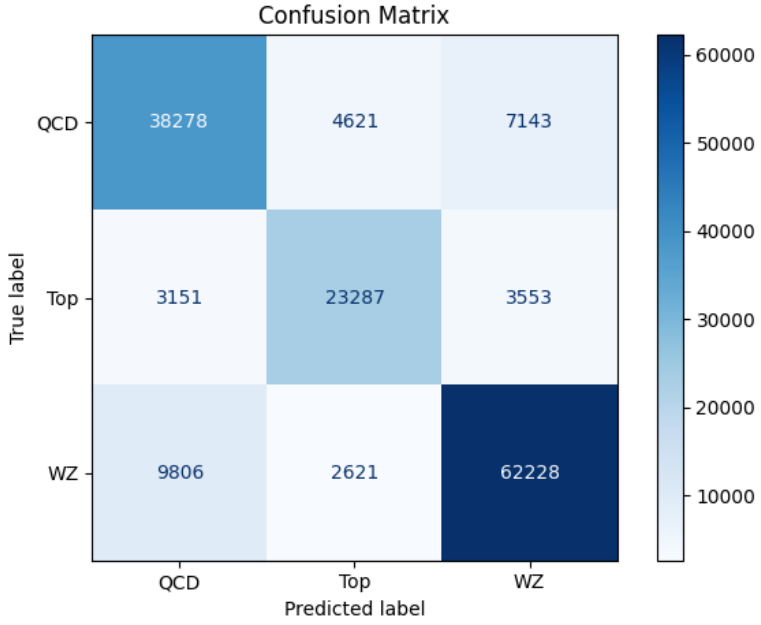


Figure 12: Confusion matrix for the best-performing multiclass model (scalar MLP). Diagonal dominance indicates strong class separation.

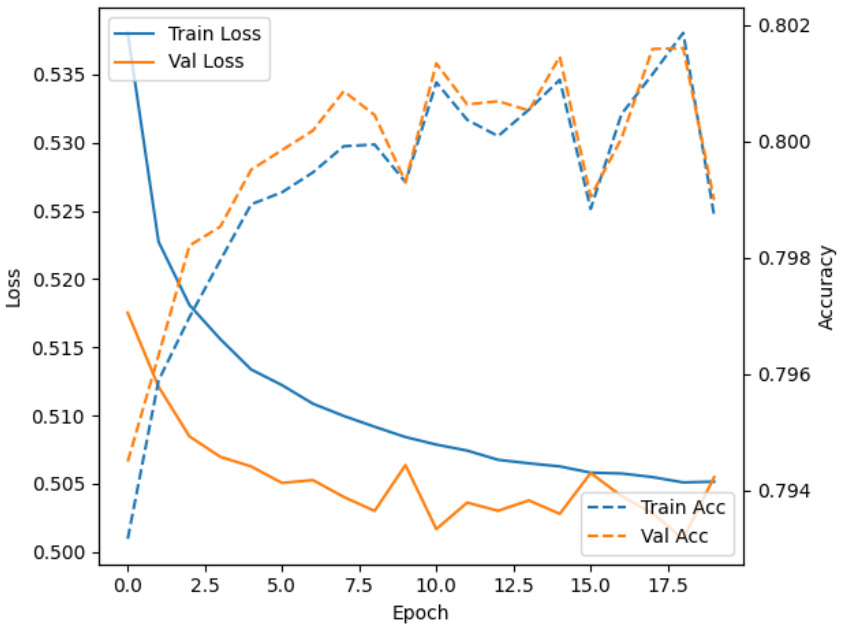


Figure 13: Training and validation loss for the scalar multiclass model. No overfitting observed.

5.1 Effect of Jet Four-Momentum

To isolate the impact of jet kinematics, the four-vector cut is again applied to the scalar dataset. Extending feature-level reweighting to the multiclass case would require separate balancing across all class-feature combinations, which is beyond the scope of this study.

The resulting model achieves a balanced accuracy of 0.760 and a weighted F1 score of 0.771. While performance is slightly lower compared to models with full four-momentum information, this configuration ensures that the classifier relies exclusively on substructure variables. The drop in accuracy highlights the significant discriminative power of jet mass and p_T, which remain entangled with class identity even in the multiclass setting.

6 Model Configuration and Optimization Space

The developed deep learning pipeline offers flexibility across numerous training parameters, including: batch size, constituent augmentation, four-vector removal, dropout, batch normalization, hidden dimension size, network architecture, learning rate, epoch count, regularization strength, class and feature weighting, and the use of scalar and/or constituent inputs.

Due to computational constraints (limited runtime and CPU resources), an exhaustive search over the full parameter space was not feasible. Therefore, the models presented here are not necessarily optimal. A broader hyperparameter sweep on GPU-accelerated systems could yield improved performance for specific classification tasks.

6.1 Network Architecture

As described in Section 3, the default architecture labeled as `standard` was used throughout this study. Alternatives included a `deep` MLP (two additional hidden layers) and a `downsized` variant (shrinking hidden dimensions across layers). For Deep Sets, `standard` and `downsized` ϕ and ρ MLPs were supported; the `deep` variant was excluded due to unmanageable training time.

Initial comparisons suggest that the `standard` configuration offered the best balance of accuracy, stability, and runtime efficiency.

6.2 Training Epochs and Regularization Effects

The number of training epochs was capped at 20 due to resource limits. Early stopping with a patience of 5 was used. For longer training runs, smaller learning rates and higher patience would allow more thorough convergence.

Dropout was effective in improving generalization but introduced variance in short training cycles. This noise diminishes with longer training 8. The effect is more pronounced in Deep Sets, which require longer convergence times due to their higher complexity. Their slightly lower performance here likely reflects early stopping before full convergence. See Fig 14.

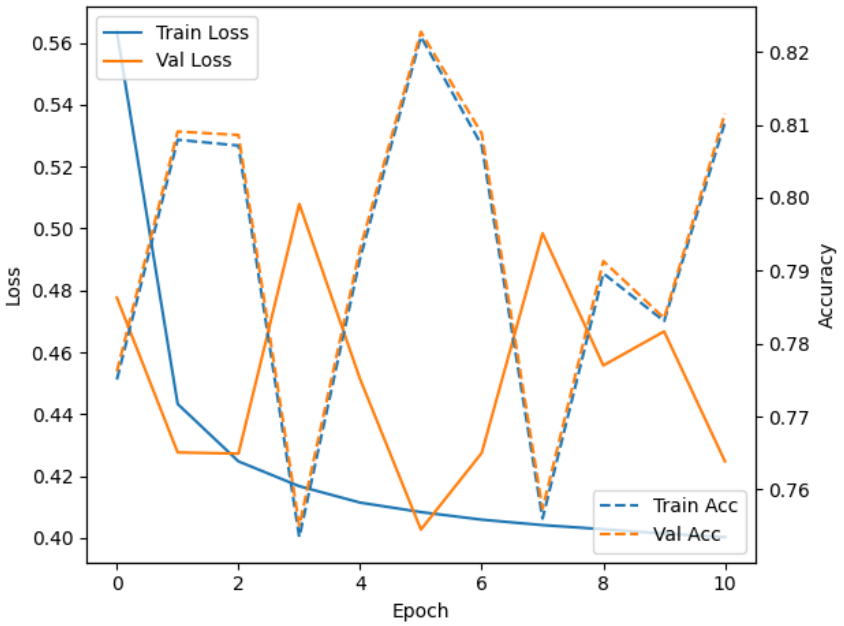


Figure 14: A high-noise training summary example for Deep Sets model in early epochs.

6.3 Generalization Tools: Dropout, BatchNorm, and Regularization

Dropout, batch normalization, and L_2 regularization were employed to prevent overfitting and encourage generalization —core goals of deep learning. However, these techniques introduce early training noise and demand more epochs to fully benefit deeper architectures like Deep Sets.

6.4 Hyperparameter Phase Space

Many parameters remain unexplored in this work. A full grid or Bayesian hyperparameter search —parallelized across multiple runs— could significantly enhance performance. Thus, the results in this paper represent a conservative baseline, and further improvements are expected with better-tuned configurations.

7 Conclusion

This work presented a flexible and modular deep learning pipeline for classifying jets in high-energy physics, using both scalar and constituent-level features. Multiple model variants were implemented and evaluated on binary and multiclass classification tasks, with careful attention to class imbalance, feature biases, and training configurations.

Scalar-based models, particularly the standard MLP, achieved the best overall performance, driven largely by strong kinematic separability. Deep Sets offered a permutation-invariant alternative using constituent-level inputs, but required longer training to reach comparable performance. Efforts to reduce kinematic dependence —via feature reweighting and four-vector removal— revealed the deep entanglement of jet substructure and jet kinematics.

7.1 Further Improvements

Future improvements can explore enriching the scalar inputs by appending Cartesian representations of four-momenta. While this duplicates kinematic information and further biases the model toward mass and p_T, it may encode additional physical structure beneficial for classification.

Another direction is to represent jets as images by projecting constituent features onto the η–ϕ plane and constructing 2D grids (jet images), as shown in Figure 18. These can be fed into convolutional neural networks (CNNs), which excel at local spatial pattern recognition. However, studies suggest that for rank-3 constituent tensors (e.g., $(N_{\text{jets}}, N_{\text{const}}, N_{\text{features}})$), Deep Sets already offer competitive performance with minimal preprocessing, and CNN gains are often marginal in this context [7].

References

[1] ATLAS Collaboration, "Jet Tagging Dataset for Machine Learning," *Unpublished teaching dataset*, provided via SWITCHdrive: https://drive.switch.ch/index.php/s/H7I57It4auYxJmb, 2024.

[2] ResearchGate, Fig.1, "A schematic diagram of an interaction producing two light-flavour jets and one b-jet," reproduced under Creative Commons Attribution 4.0. Retrieved from ResearchGate:contentReference[oaicite:2]index=2.

[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016. Chapter 6.

[4] N. Srivastava *et al.*, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[5] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. of ICML*, 2015. https://arxiv.org/abs/1502.03167

[6] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. Smola, "Deep Sets," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. https://arxiv.org/abs/1703.06114

[7] P. T. Komiske, E. M. Metodiev, and J. Thaler, "Energy Flow Networks: Deep Sets for Particle Jets," in *Journal of High Energy Physics (JHEP)*, vol. 2019, no. 1, p. 121, 2019. https://arxiv.org/abs/1810.05165

8 Appendix

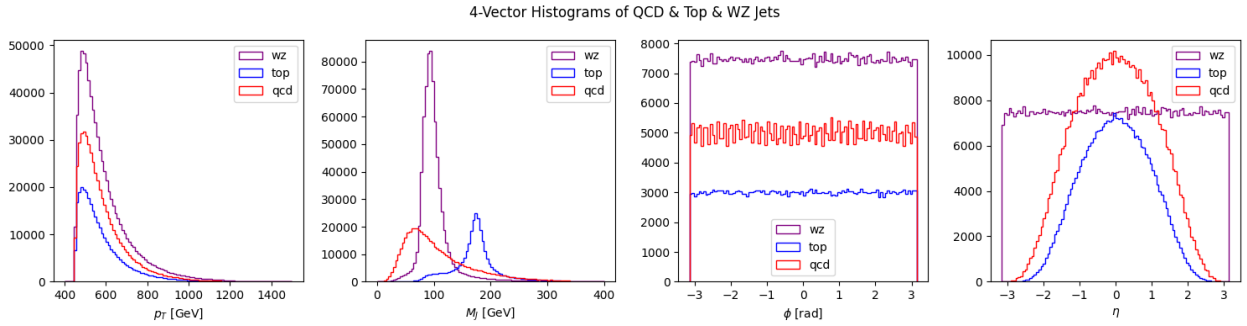


Figure 15: Histogram distributions of 4-vector features of the QCD, Top, & W/Z jets. In the order of p_T, m, ϕ, η.

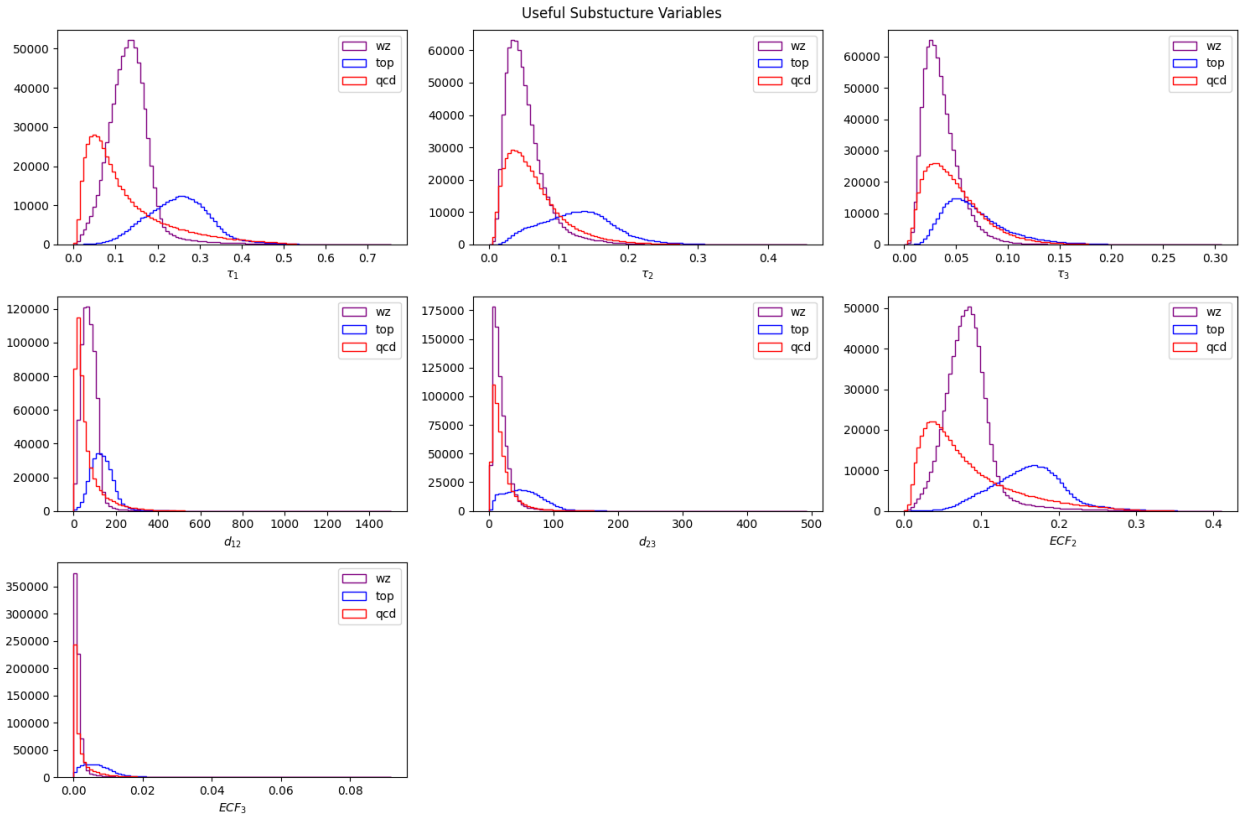


Figure 16: Histogram distributions of substructure features of the QCD, Top, & W/Z jets. In the order of τ_1, τ_2, τ_3, d_{12}, d_{23}, ECF_2, ECF_3.

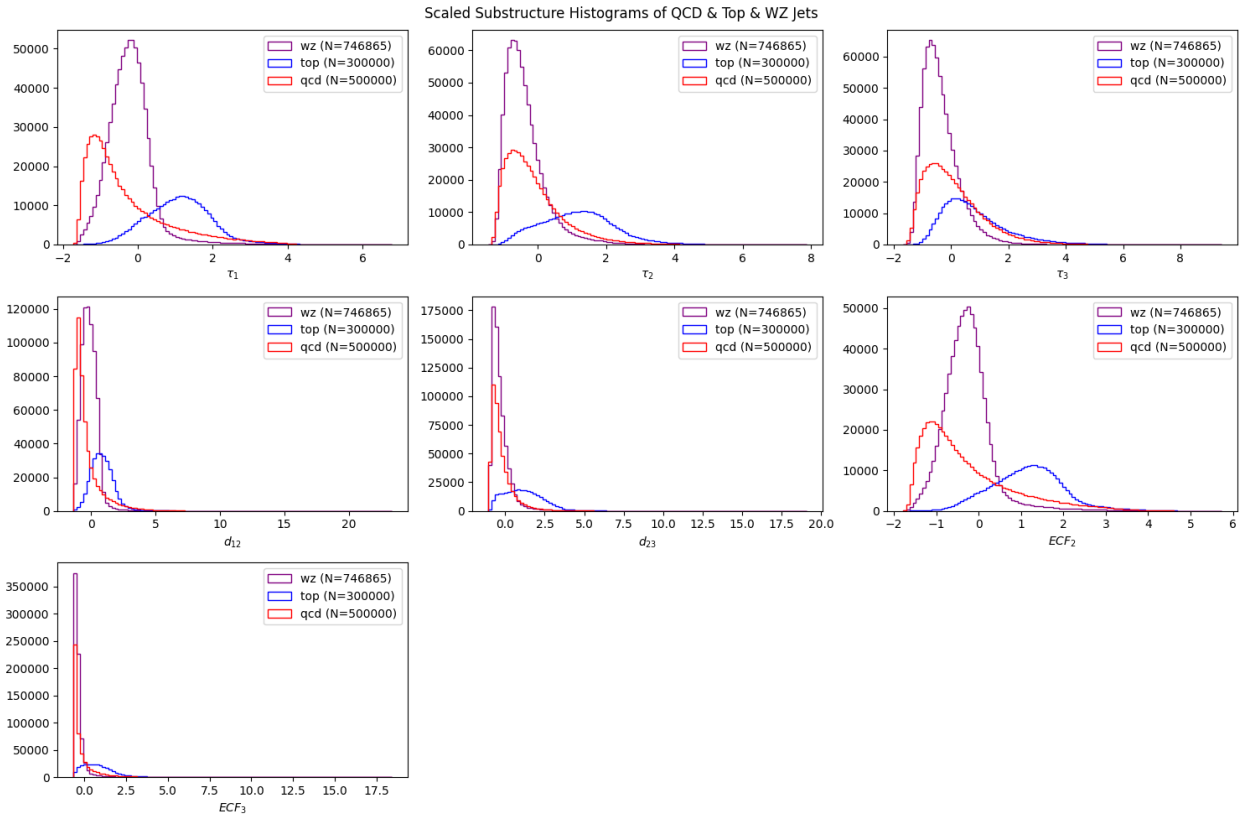


Figure 17: Histogram distributions of scaled substructure features of the QCD, Top, & W/Z jets. In the order of τ_1, τ_2, τ_3, d_{12}, d_{23}, ECF_2, ECF_3.

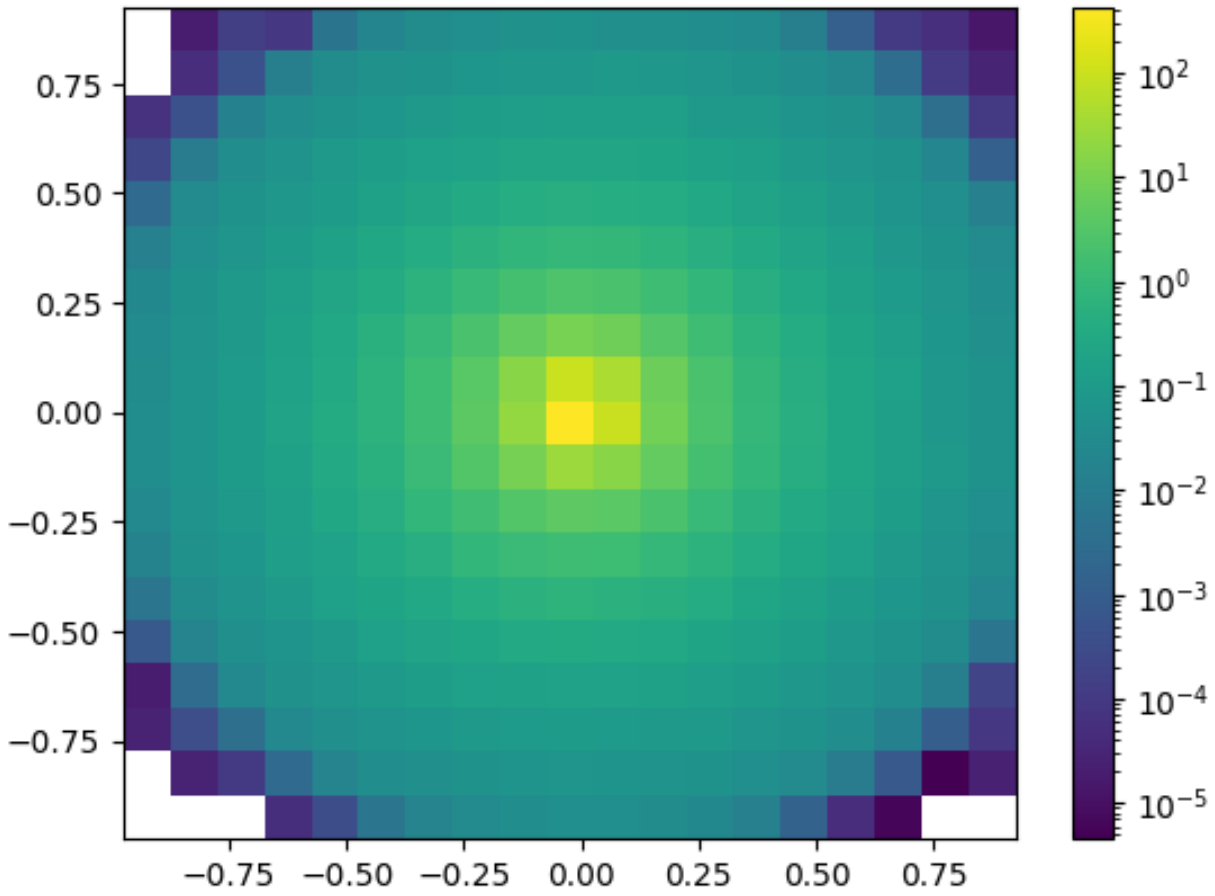


Figure 18: Example jet image made using the new calculated E of constituents.